DESENVOLVIMENTO MOBILE

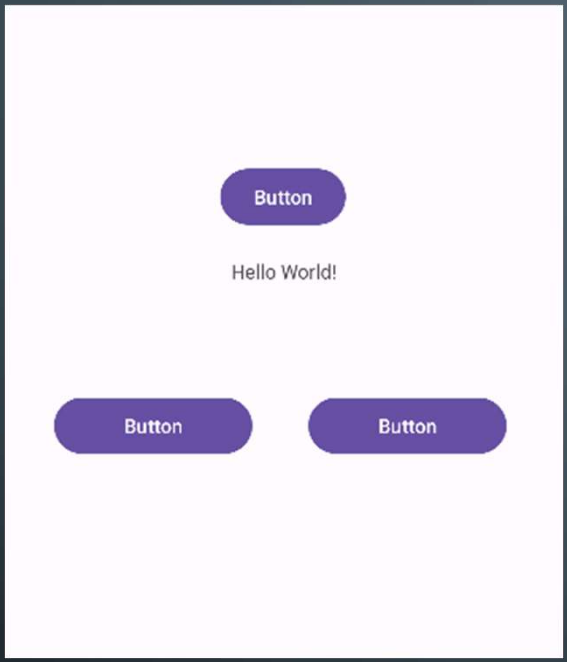
ALEX ZACHARIAS

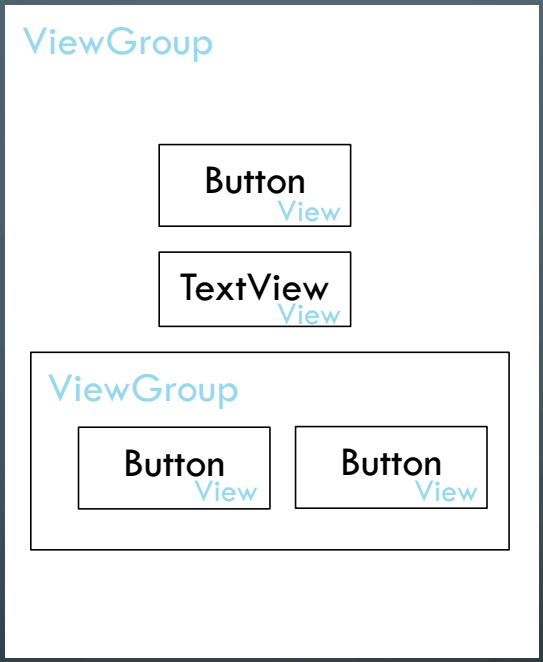
ALEX@EINSTEINLIMEIRA.COM.BR

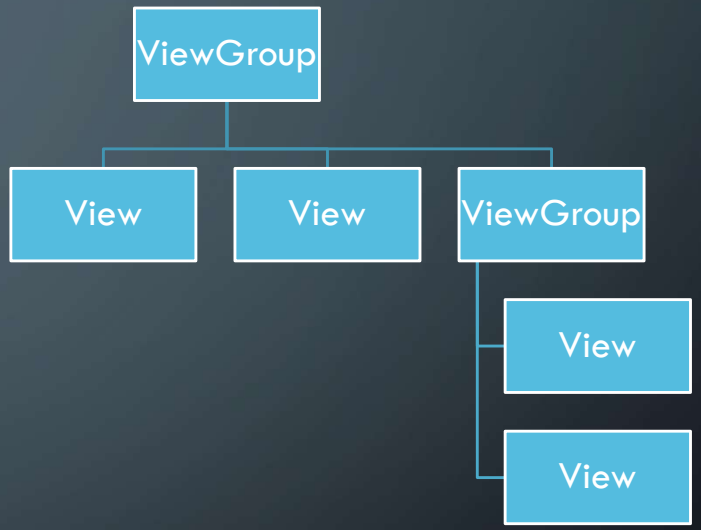
ANDROID - LAYOUT

- Em Android, seja utilizando Java ou Kotlin o layout fica separado da codificação do aplicativo:
  - Temos arquivos java (.java) ou kotlin (.kt) contendo toda a logica de seu aplicativo
  - A parte visual (o layout) é feito em arquivos .xml
- Qualquer elemento dentro do layout segue a hierarquia de View e ViewGroup:
  - View: elementos da interface que o usuário interage
  - ViewGroup: elementos que agrupam outros elementos

ANDROID - LAYOUT







- Todo layout precisa começar com um componente do tipo ViewGroup

ANDROID - LAYOUT

- O Layout define a estrutura visual da interface do usuário.
- É composto por um ou mais ViewGroup/Views, que juntos formam o design do aplicativo e interagem com o usuário.
- Os padrões de layout mais comuns são:
- LinearLayout: organiza suas views em uma única linha vertical ou horizontal.

ANDROID - LAYOUT

- RelativeLayout: Permite alinhar views uma em relação a outra ou em relação ao pai, o ViewGroup onde sua view foi colocada.

- ConstraintLayout: uma forma de desenhar layouts complexos sem a necessidade de usar hierarquias agrupadas, ou seja, permite criar views de uma forma muito mais flexível, sem precisar agrupar layouts uns dentro dos outros.

ANDROID - LINEARLAYOUT

- Vamos começar utilizando o LinearLayout.
- Quando criamos um projeto novo no Android Studio ele cria automaticamente com o ConstraintLayout.
- Vamos criar um projeto padrão em branco.
- Exiba o arquivo activity_main.xml

activity_main.xml

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.and
3      xmlns:app="http://schemas.android.com/apk/res-auto"
4      xmlns:tools="http://schemas.android.com/tools"
5      android:layout_width="match_parent"
```

ANDROID - LINEARLAYOUT

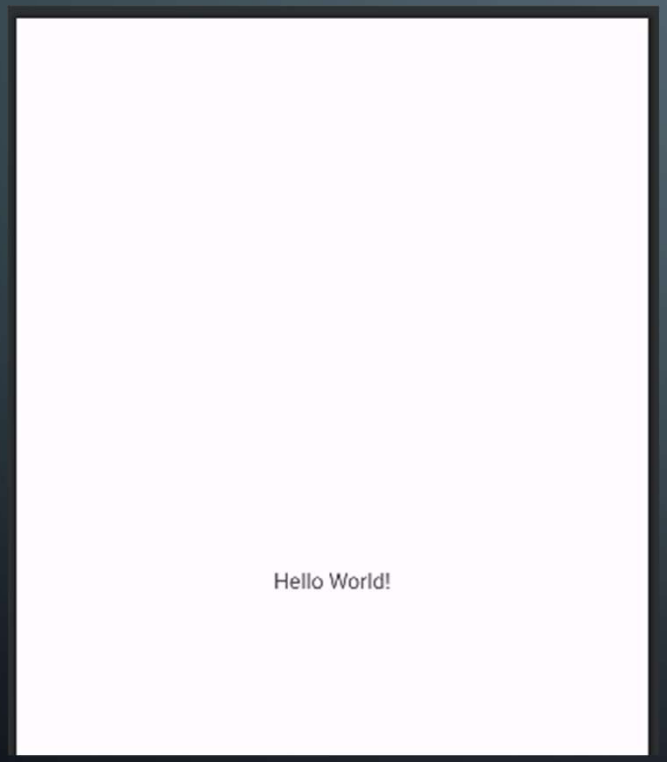
- Troque a tag ConstraintLayout para LinearLayout:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/a
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</LinearLayout>
```

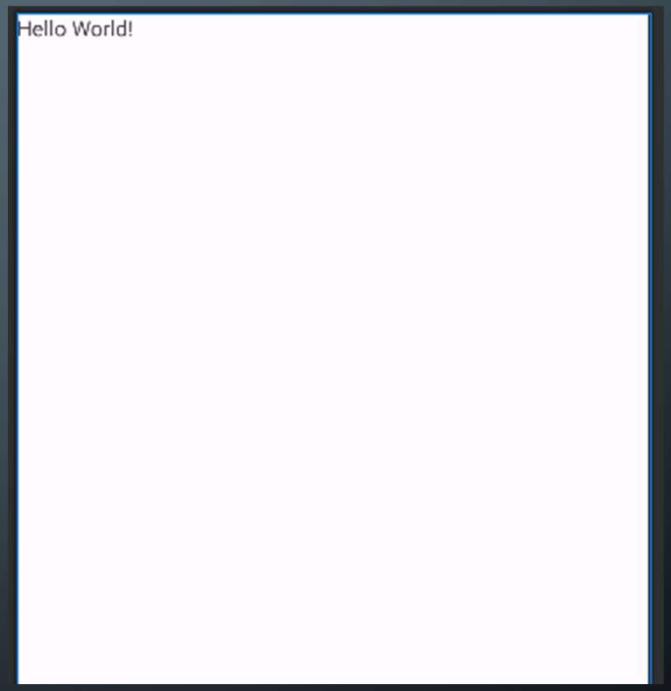
- Note que ele altera automaticamente a tag no final do arquivo para LinearLayout

ANDROID - LINEARLAYOUT

ConstraintLayout:



LinearLayout:



ANDROID - LINEARLAYOUT

- Vamos apagar os comandos que ele criou automaticamente no TextView, deixando apenas as linhas abaixo:

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!" />
```

- layout_width: largura do componente
- layout_height: altura do componente
- text: texto que será exibido na tela

ANDROID - LINEARLAYOUT

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello World!" />
```

- wrap_content: ajusta o tamanho do componente baseado no espaço necessário para que seu conteúdo (o texto nesse caso) caiba nele
- match_parent: ajusta o componente em relação ao componente pai.

ANDROID - LINEARLAYOUT

- Alterando apenas a largura do componente:

wrap_content:

match_parent:

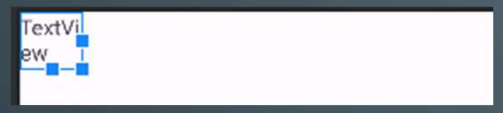
- A linha azul indica o tamanho do componente na tela, note as diferenças.
- Na prática só veríamos diferença na execução do aplicativo se mudarmos a cor do fundo do componente ou se tentarmos colocar um componente ao lado dele.

ANDROID - LINEARLAYOUT

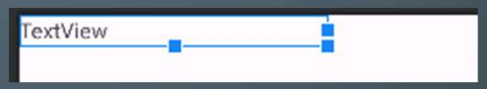
- Podemos alterar também o tamanho do componente informando a medida que queremos utilizar nos métodos layout_width ou layout_height, entre as varias escalas de tamanho diferentes que o Android possui, para tamanho de componentes na tela o melhor a se usar é o dp (Density-Independent Pixels), essa é a melhor escala pois faz com que o componente tenha sempre o mesmo tamanho independente da resolução da tela.

ANDROID - LINEARLAYOUT

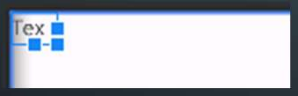
```
<TextView
    android:layout_width="40dp"
    android:layout_height="wrap_content"
    android:text="TextView" />
```




```
<TextView
    android:layout_width="200dp"
    android:layout_height="wrap_content"
    android:text="TextView" />
```




```
<TextView
    android:layout_width="30dp"
    android:layout_height="20dp"
    android:text="TextView" />
```



- Note que se colocarmos valores fixo tanto para a largura, quanto para a altura e não tiver espaço suficiente para colocar o texto, o mesmo será cortado.

ANDROID - LINEARLAYOUT

- Para mudar o tamanho da fonte utilizamos o comando textSize, passando para ele o numero seguido das letras sp, essa sigla define qual escala deve ser utilizada.
- O Android tem vários padrões de escala para definir tamanho dos componentes, para texto devemos sempre utilizar o sp (scale-independent pixels) que independentemente do tamanho da tela, ele sempre reproduz as letras do mesmo tamanho.
- Por exemplo: utilizar 12sp faz com que as letras sempre exibam no tamanho correspondente as fontes de tamanho 12.

ANDROID - LINEARLAYOUT

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Tamanho 12"
    android:textSize="12sp"/>
```

Tamanho 12

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Tamanho 20"
    android:textSize="20sp"/>
```

Tamanho 20

- Agora vamos colocar mais componentes na tela, a principio vamos utilizar vários textView.
- Note no próximo slide que o LinearLayout faz com que esses componentes fiquem um na frente do outro:

ANDROID - LINEARLAYOUT

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="TextView 1"
    android:textSize="20sp" />

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="TextView 2"
    android:textSize="20sp" />

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="TextView 3"
    android:textSize="20sp" />
```

TextView 1TextView 2TextView 3

- Os componentes ficam assim porque o LinearLayout faz com que todos sejam colocados de forma sequencial, e por padrão ele vem no formato horizontal.

ANDROID - LINEARLAYOUT

- Para trabalhar com essas posições podemos utilizar o atributo orientation do componente LinearLayout.

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/a
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
```

TextView 1
TextView 2
TextView 3

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res,
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="horizontal"
    tools:context=".MainActivity">

    <TextView
        android:layout_width="wrap_content"
```

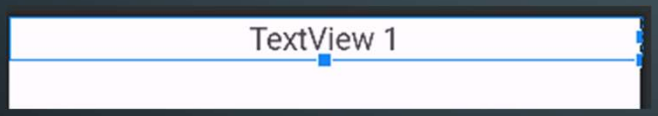
TextView 1TextView 2TextView 3

- Se não informar o orientation ele fica como horizontal

ANDROID - LINEARLAYOUT

- Se colocar um componente único utilizando toda a largura da tela (match_parent) mas desejamos centralizar seu texto, devemos utilizar gravity:

```
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="TextView 1"
    android:gravity="center"
    android:textSize="20sp" />
```




```
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="TextView 1"
    android:gravity="end"
    android:textSize="20sp" />
```

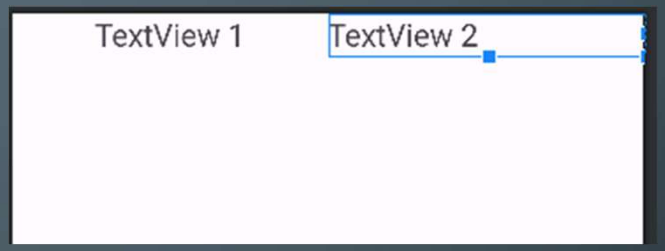
- Para alinhar a direita devemos utilizar end e a esquerda start, para que o app funcione caso utilize línguas que escrevem da direita para a esquerda.

ANDROID - LINEARLAYOUT

- Podemos fazer com que o Android distribua toda a largura da tela em partes iguais (ou diferentes) entre os elementos que queremos exibir.
- Vamos deixar dois elementos na tela e dar peso para eles, forçando a divisão por igual entre eles.
- Nessa caso valos utilizar o atributo layout_weight que faz com que o Android some todos os valores atribuídos e divida entre os componentes.
- Para utilizar o layout_weigth devemos zerar o layout_width

ANDROID - LINEARLAYOUT

```
<TextView
    android:layout_width="0dp"
    android:layout_weight="1"
    android:layout_height="wrap_content"
    android:text="TextView 1"
    android:gravity="center"
    android:textSize="20sp" />

<TextView
    android:layout_width="0dp"
    android:layout_weight="1"
    android:layout_height="wrap_content"
    android:text="TextView 2"
    android:textSize="20sp" />
```

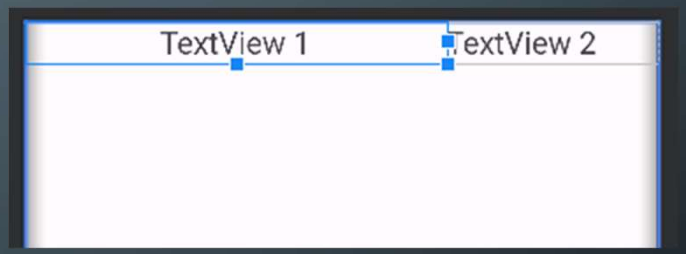


- O primeiro textView esta centrado e o segundo não, mas ele dividiu a tela em partes iguais entre ambos componentes.

ANDROID - LINEARLAYOUT

- Podemos dividir em partes diferentes apenas mudando o valor (peso) que utilizamos na propriedade layout_weight.

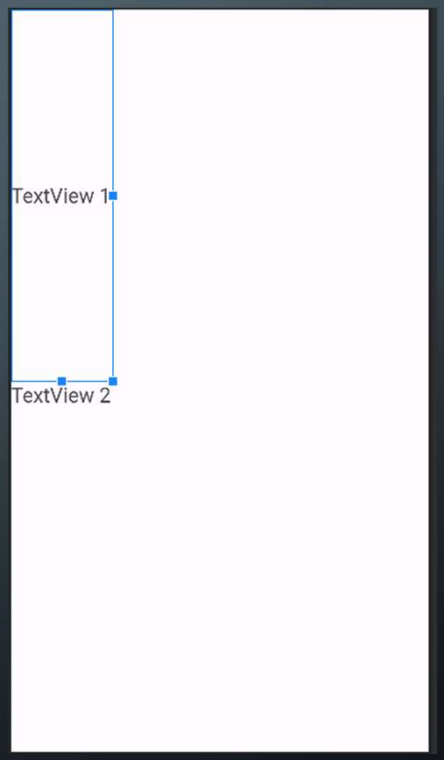
```
<TextView
    android:layout_width="0dp"
    android:layout_weight="2"
    android:layout_height="wrap_content"
    android:text="TextView 1"
    android:gravity="center"
    android:textSize="20sp" />

<TextView
    android:layout_width="0dp"
    android:layout_weight="1"
    android:layout_height="wrap_content"
    android:text="TextView 2"
    android:textSize="20sp" />
```



ANDROID - LINEARLAYOUT

- Essa definição de tamanho também funciona na vertical, basta zerar a altura e definir o LinearLayout na vertical:

```
android:orientation="vertical"
tools:context=".MainActivity">

<TextView
    android:layout_width="wrap_content"
    android:layout_weight="1"
    android:layout_height="0dp"
    android:text="TextView 1"
    android:gravity="center"
    android:textSize="20sp" />

<TextView
    android:layout_width="wrap_content"
    android:layout_weight="1"
    android:layout_height="0dp"
    android:text="TextView 2"
    android:textSize="20sp" />
```



ANDROID - CORES

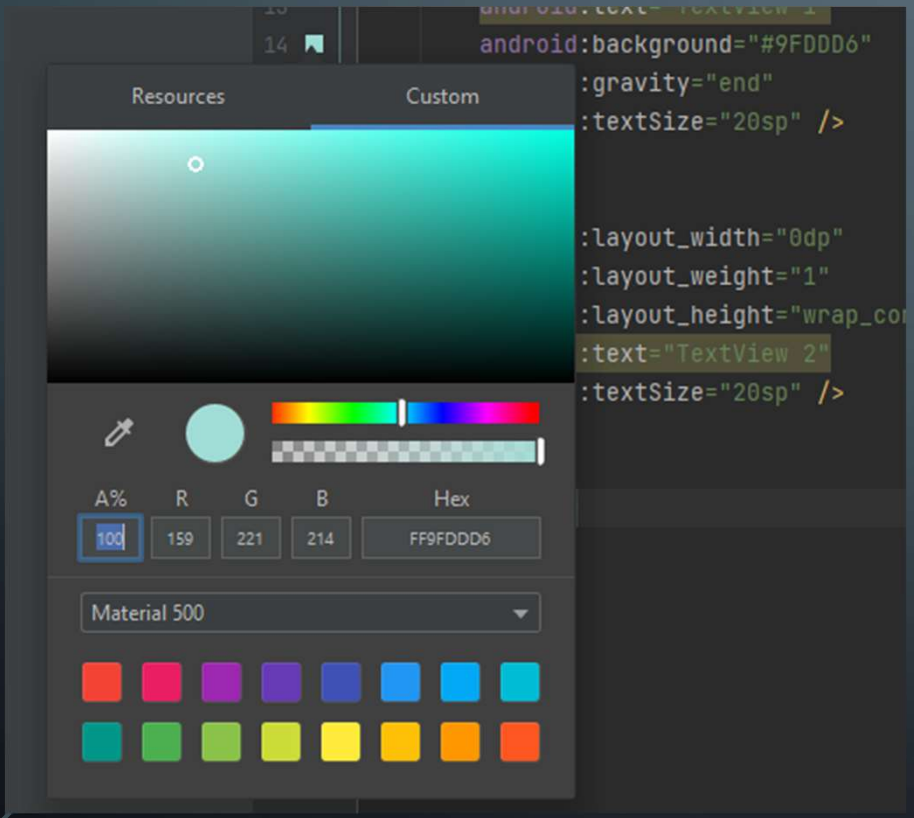
- Vamos alterar a cor de fundo do objeto com o comando background, podemos utilizar uma cor já definida no nosso projeto ou utilizar o código hexa da cor desejada iniciando com #.

```
10    <TextView
11        android:layout_width="wrap_content"
12        android:layout_height="wrap_content"
13        android:text="TextView 1"
14        android:background="#fff"
15        android:gravity="end"
16        android:textSize="20sp" />
17
```

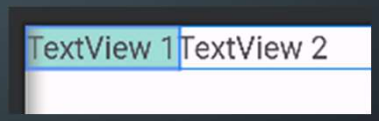
- Note que ao colocar #fff ele aparece um quadrado branco a esquerda da linha, clique nele para selecionar a cor desejada, isso é muito útil quando você não sabe o código hexa da cor que deseja utilizar.

ANDROID - CORES

- Basta selecionar a cor desejada na janela exibida:

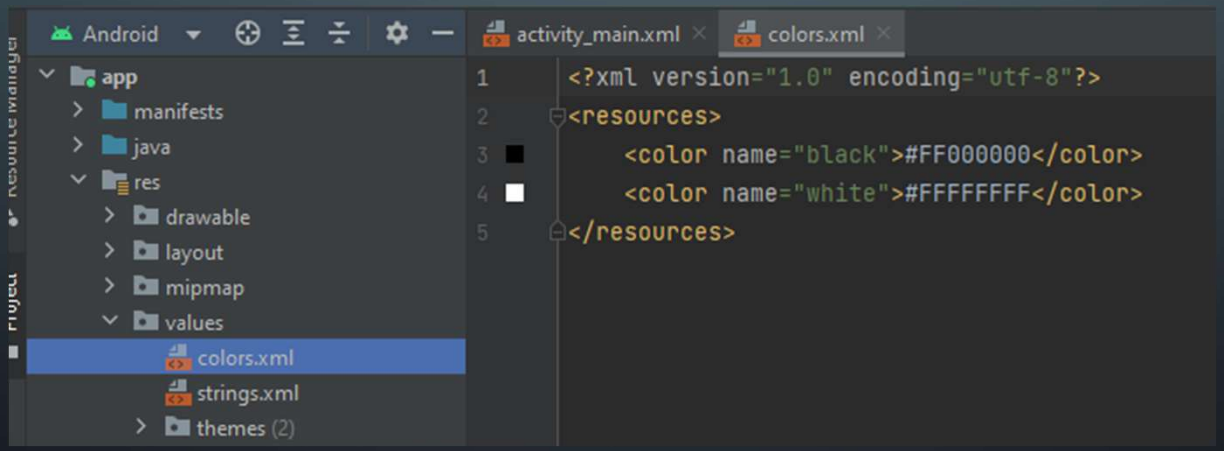



```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="TextView 1"
    android:background="#9FDDD6"
    android:gravity="end"
    android:textSize="20sp" />
```



ANDROID - CORES

- Ao invés de utilizar diretamente o código hexa das cores, podemos definir todas as cores do projeto dentro do arquivo colors.xml, isso é muito útil se precisarmos trocar as cores do projeto, facilitando muito essa etapa.



ANDROID - CORES

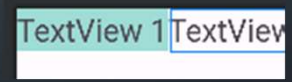
- Vamos criar uma cor com o nome fundo_texto e colocar uma cor padrão nele:

activity_main.xml | colors.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
    <color name="fundo_texto">#9FDDD6</color>
</resources>
```

- Para utilizar essa cor devemos colocar @color/fundo_texto no componente desejado:

```xml
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="TextView 1"
    android:background="@color/fundo_texto"
    android:textSize="20sp" />
```



ANDROID - CORES

- O atributo background pode ser utilizado praticamente em qualquer componente, inclusive no ViewGroup principal para mudar a cor do fundo da aplicação.
- Para trocar a cor do texto devemos utilizar o atributo textColor, seguindo o mesmo padrão do background, usar cor pré definida no projeto ou o código hexa da cor.
- Aconselho a utilizar as cores pré definidas no arquivo colors.xml.

ANDROID - CORES

activity_main.xml | colors.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="black">#FF000000</color>
    <color name="white">#FFFFFFFF</color>
    <color name="fundo_texto">#9FDDD6</color>
    <color name="texto_ver">#F44336</color>
</resources>
```

```xml
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="TextView 1"
    android:background="@color/fundo_texto"
    android:textColor="@color/texto_ver"
    android:textSize="20sp" />
```

TextView 1 TextView 2

ANDROID - MARGENS

- Temos dois atributos principais para trabalhar com as margens:
- layout_margin: para alterar as margens do componente em relação ao seu ViewGroup (no nosso caso o LinearLayout).
- padding: altera a margem interna do componente, entre seu conteúdo (texto no nosso caso) e sua margem.
- E cada um deles pode ser definido só de um lado, só do outro, encima, embaixo, em todos os lados...

ANDROID - MARGENS

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="TextView 1"
    android:background="@color/fundo_texto"
    android:textColor="@color/texto_ver"
    android:textSize="20sp" />
```

TextView 1

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:padding="16dp"
    android:text="TextView 1"
    android:background="@color/fundo_texto"
    android:textColor="@color/texto_ver"
    android:textSize="20sp" />
```

TextView 1

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_margin="16dp"
    android:text="TextView 1"
    android:background="@color/fundo_texto"
    android:textColor="@color/texto_ver"
    android:textSize="20sp" />
```

TextView 1

```
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:padding="16dp"
    android:layout_margin="16dp"
    android:text="TextView 1"
    android:background="@color/fundo_texto"
    android:textColor="@color/texto_ver"
    android:textSize="20sp" />
```

TextView 1

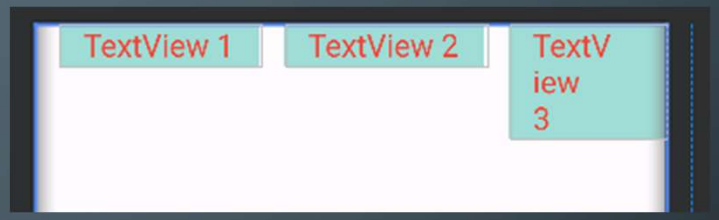
ANDROID - MARGENS

- Podemos utilizar o layout_margin ou o padding apenas no lado que desejamos ou a combinação de dois ou mais lados. Para isso utilizamos os comandos abaixo:
- layout_marginStart ou paddingStart: a esquerda
- layout_marginEnd ou paddingEnd: a direita
- layout_marginTop ou paddingTop: na parte superior
- layout_marginBotton ou paddingBotton: na parte inferior
- layout_marginHorizontal ou paddingHorizontal: a direita e esquerda
- layout_marginVertical ou paddingVertical: superior e inferior

ANDROID – ROLAGEM DE TELA

- Se colocarmos vários componentes, vai chegar uma hora que eles não caberão mais na tela:

```
10  <TextView
11      android:layout_width="wrap_content"
12      android:layout_height="wrap_content"
13      android:paddingHorizontal="16dp"
14      android:layout_marginStart="16dp"
15      android:text="TextView 1"
16      android:background="@color/fundo_texto"
17      android:textColor="@color/texto_ver"
18      android:textSize="20sp" />
19  <TextView
20      android:layout_width="wrap_content"
21      android:layout_height="wrap_content"
22      android:paddingHorizontal="16dp"
23      android:layout_marginStart="16dp"
24      android:text="TextView 2"
25      android:background="@color/fundo_texto"
26      android:textColor="@color/texto_ver"
27      android:textSize="20sp" />
28  <TextView...>
37  <TextView...>
46
```



- Note que o 3° TextView ficou apertado no final da tela, o 4° nem apareceu.
- Obs: podemos clicar no sinal de – ou + existente a esquerda do componente para contrair ou expandir o código.
  - Os dois últimos TextView estão contraídos no código ao lado

ANDROID – ROLAGEM DE TELA

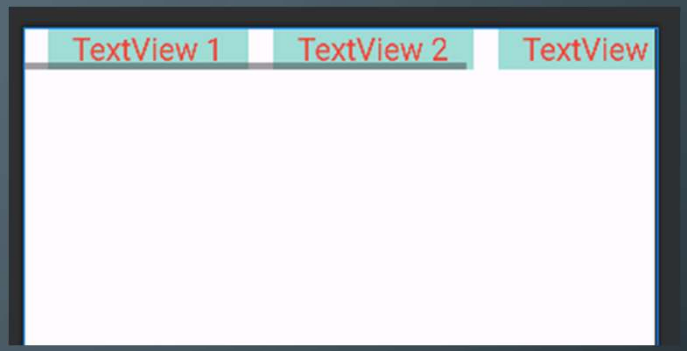
- Para resolver esse problema, podemos utilizar os componentes:
  - ScrollView: faz a rolagem da tela na vertical.
  - HorizontalScrollView: faz a rolagem da tela na horizontal.

```
<HorizontalScrollView
    android:layout_width="match_parent"
    android:layout_height="wrap_content" >

//Colocar seu componente aqui

</HorizontalScrollView>
```

- Mas esses componentes só aceitam um componente abaixo deles, por isso devemos utilizar um ViewGroup., no caso um Linearlayout.

ANDROID – ROLAGEM DE TELA

```
<HorizontalScrollView
    android:layout_width="match_parent"
    android:layout_height="wrap_content" >

<LinearLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content">

    <TextView...>
    <TextView...>
    <TextView...>
    <TextView...>

</LinearLayout>

</HorizontalScrollView>
```



- Note a barra de rolagem abaixo dos componentes, você só conseguira rodar se executar o programa.